



Coordinación de
Educación Abierta y a Distancia
VICERRECTORADO ACADÉMICO



ESTRUCTURAS DE DATOS

Actividad Autónoma

Nombre de la actividad Autónoma:

Sistema de Rutas Aéreas EcoVuelo

Unidad 4: Estructuras de Datos No Lineales

Tema 1: Grafos



FACULTAD DE
Ingeniería

Nombres:	Freddy Anibal Erazo Guerrero
Fecha:	26 de Julio de 2026
Carrera:	Ingeniería en Ciencia de Datos e Inteligencia Artificial
Periodo académico:	2026 - 1S
Semestre:	Segundo C

Objetivo de la actividad:

Implementar un sistema interactivo en Python que modele una red logística aérea real mediante un Grafo, aplicando de forma correcta sus operaciones fundamentales de gestión, consulta y recorrido (BFS/DFS).

Recursos o temas que debe haber estudiado antes de hacer la actividad:

Tema 1: Grafos

- 1.1 Definición y tipos de árboles binarios
- 1.2 Operaciones de árboles binarios
- 1.3 Representación de un árbol binario
- 1.4 Árbol de búsqueda binario

Formato de entrega: PDF

Instrucciones:

- Mantener orden y claridad en la elaboración de la tarea.
- Subir un único documento PDF al aula virtual.
- Hacer la actividad de forma individual
- Nombrar al archivo de la siguiente manera: Apellido_Nombre_Unidad4_Tema1.
- Si se detecta plagio, la nota de la actividad será de cero.
- Asegurarse que el contenido del documento pueda observarse de manera clara.

Contenido:

Contexto de la tarea:

Sistema de Rutas Aéreas EcoVuelo

Usted ha sido contratado como ingeniero de datos para EcoVuelo, una nueva aerolínea de bajo costo enfocada en la eficiencia energética. El objetivo del proyecto es desarrollar el motor lógico que gestione la red de transporte de la compañía.

Para este sistema:

- Los Vértices (Nodos) representan los Aeropuertos / Ciudades.
- Las Aristas (Arcos) representan las Rutas de vuelo directo entre ciudades.
- El Peso de cada arista representará la distancia en kilómetros (km) o el consumo de combustible entre ambos destinos.

Regla obligatoria

Cada estudiante deberá elegir una región geográfica real del país (ej. Costa, Sierra, Oriente, etc.) con un mínimo de 6 ciudades reales.

Además, deberá definir desde el inicio si su red será Dirigida o No Dirigida, justificando esta decisión técnica en su informe. El grafo será ponderado de forma obligatoria.

No se aceptarán proyectos con la misma configuración de ciudades entre compañeros.

Requerimientos Técnicos

El sistema debe programarse en Python y permitir al usuario interactuar con el grafo a través de las siguientes 10 operaciones obligatorias:

1. Añadir Ciudad (Agregar Vértice): Permite insertar un nuevo aeropuerto al sistema.
2. Eliminar Ciudad (Eliminar Vértice): Elimina un aeropuerto de la red. Nota: Al eliminar la ciudad, se deben eliminar automáticamente todas las rutas (aristas) conectadas a ella para evitar conexiones huérfanas.
3. Añadir Ruta (Agregar Arista Ponderada): Crea un vuelo directo entre dos ciudades existentes asignándole su respectivo peso (distancia en km).
4. Eliminar Ruta (Eliminar Arista): Cancela un vuelo directo entre dos ciudades, removiendo la arista correspondiente.
5. Buscar Ruta (Verificar Arista): Comprueba e informa si existe o no un vuelo directo entre la Ciudad A y la Ciudad B.
6. Buscar Ciudad (Verificar Vértice): Comprueba si un aeropuerto específico ya está registrado en el sistema.
7. Destinos Directos (Obtener Adyacentes): Muestra la lista de todas las ciudades conectadas directamente desde un aeropuerto de origen.
8. Consultar Distancia (Obtener Peso): Devuelve el valor numérico (peso) de la ruta directa entre dos ciudades.
9. Calcular Conectividad (Grado del Vértice): Calcula cuántas rutas totales se conectan a una ciudad específica. (Si se elige un grafo dirigido, se debe diferenciar entre vuelos de salida e ingresos).
10. Exploración de la Red (Recorridos):
 - Implementar BFS (Búsqueda en Anchura) para listar los destinos visitados nivel por nivel desde una ciudad inicial.
 - Implementar DFS (Búsqueda en Profundidad) para explorar las rutas lo más lejos posible antes de retroceder, verificando la conectividad de la red.

3. Interfaz de Usuario y Librerías

Existe libertad absoluta para el diseño de la interfaz y la visualización de los datos. Puede elegir cualquiera librería para la parte de la interfaz.

Entregable:

- Formato: Un único archivo PDF.
- Incluir en el archivo un enlace público al código alojado en la nube de preferencia GitHub.
- El documento debe ser autosuficiente. Si debo abrir su código para entender qué hicieron, el informe no está bien redactado. El PDF debe explicar el proyecto de principio a fin por sí solo.

Bibliografía:

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009) *Introduction to Algorithms* (3rd ed.). MIT Press.
- Sahni, S. (2005). *Data Structures, Algorithms, and Applications in C++* (2nd ed.). Universities Press.
- Lafore, R. (2002). *Data Structures and Algorithms in Java* (2nd ed.). Sams Publishing.
- Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Python*. Wiley.

Rúbrica de evaluación

Componente de aprendizaje:	Autónomo	X	Contacto con el Docente
Resultado(s) de aprendizaje:	- Diseña grafos, aplica algoritmos de búsqueda y análisis en grafos, y utiliza estas estructuras en aplicaciones prácticas. - Aplica estrategias eficientes de búsqueda en diferentes contextos, demostrando comprensión de algoritmos como búsqueda binaria y búsqueda en estructuras no ordenadas. - Analiza algoritmos de ordenación, comprendiendo las diferencias entre métodos simples y eficientes, y aplicando este conocimiento en la práctica.		
Nombre de la Unidad:	Unidad 4: Estructuras de Datos No Lineales		
Nombre de la Actividad:	Actividad autónoma Unidad 4 Tema 1		

Criterios de Evaluación	Escala de Valoración						
	Excelente (10 - 9,1)	Bueno (9 - 8,1)	Satisfactorio (8 - 7)	Necesita mejorar (6,9 - 0,1)	No entrega (0)	Puntaje	Comentarios (SIGEA)
Modelado y Personalización del Grafo	El grafo está correctamente definido según las reglas elegidas (Dirigido/No Dirigido y Ponderado). Implementa la región geográfica real con un mínimo de 6 ciudades y los pesos (km) son coherentes.	El grafo define la región geográfica y las 6 ciudades, pero carece de coherencia estricta en los pesos o tiene inconsistencias menores en la lógica del tipo de grafo (dirigido/no dirigido).	El grafo modela menos de 6 ciudades o no se evidencia una personalización geográfica real, limitándose a datos abstractos o genéricos.	El grafo no define pesos, no se adapta a ninguna región geográfica o la estructura inicial es incorrecta y no compila.	No entrega		
Lógica de Operaciones y Consultas	Las 9 operaciones de gestión y consulta funcionan perfectamente, validando errores (ej. evitar rutas huérfanas al eliminar ciudades, controlar si el usuario busca nodos que no existen).	Las operaciones funcionan en su mayoría, pero presenta fallos menores en validaciones de error (ej. el programa se cae si se ingresa una ciudad inexistente o quedan aristas huérfanas).	Solo se implementa correctamente la mitad de las operaciones de gestión y consulta, o la lógica presenta errores que alteran los datos del grafo.	Las operaciones de inserción, eliminación y consulta no funcionan, confunden nodos con aristas o causan fallos estructurales graves.	No entrega		
Algoritmos de Recorrido e Interfaz	Implementa con total exactitud los algoritmos BFS y DFS sobre la estructura elegida. La interfaz es interactiva, libre de errores y fácil de usar.	Implementa ambos recorridos, pero la interfaz presenta pequeños fallos de uso. O bien, la interfaz es excelente.	Los recorridos (BFS/DFS) muestran resultados incorrectos o incompletos. La interfaz es plana, difícil de navegar o carece de	No se implementaron los algoritmos de recorrido, o la interfaz no permite interactuar con las funciones del programa de	No entrega		

	(consola validada, NetworkX o Streamlit).	pero uno de los recorridos (BFS o DFS) tiene un error de lógica.	validaciones básicas.	forma autónoma.			
Calidad del Informe PDF (Autosuficiencia)	El PDF se explica solo. Incluye capturas claras del código y la consola. El enlace al código funciona y el diseño es profesional.	El PDF es entendible pero requiere esfuerzo para seguir la lógica. Las capturas son parciales o el enlace tiene problemas de acceso.	El PDF es solo una copia del código. No hay explicaciones del proceso o faltan las capturas de la ejecución final.	Informe deficiente	No entrega		
Innovación en La solución de la tarea (Propone un valor agregado a la actividad)	Conecta teoría y práctica con propuestas innovadoras que agregan valor a la tarea	Conecta parcialmente teoría y práctica con propuestas innovadoras que agregan valor a la tarea	Conecta teoría y práctica pero no agrega propuestas innovadoras de valor en la tarea	No propone mejoras	No entrega		

Puntaje total

Los criterios 4 y 5 están alineados a los ejes de formación del Modelo Educativo UNACH "Introspección y Prospectiva" y responden principalmente a dos de los siguientes ejes:

1. Ambiente;
2. Autonomía y adaptabilidad;
3. Comunicación;
4. Desarrollo humano;
5. Ética y valores;
6. Emprendimiento;
7. Inter y multidisciplinariedad;
8. Innovación;
9. Inclusión e interculturalidad;
10. Investigación;
11. Impacto social;
12. Tecnologías.



RESOLUCIÓN DE TAREA

Objetivo de la actividad

Implementar un sistema interactivo en Python que modele una red logística aérea real mediante un Grafo, aplicando de forma correcta sus operaciones fundamentales de gestión, consulta y recorrido (BFS/DFS).

Contexto de la tarea: Sistema de Rutas Aéreas EcoVuelo

Se ha sido contratado como ingeniero de datos para EcoVuelo, una nueva aerolínea de bajo costo enfocada en la eficiencia energética. El objetivo del proyecto es desarrollar el motor lógico que gestione la red de transporte de la compañía.

Para este sistema:

- Los Vértices (Nodos) representan los Aeropuertos / Ciudades.
- Las Aristas (Arcos) representan las Rutas de vuelo directo entre ciudades.
- El Peso de cada arista representa la distancia en kilómetros (km) entre ambos destinos.

Tipo de grafo elegido: No Dirigido Ponderado

Se eligió un grafo No Dirigido porque las rutas aéreas de EcoVuelo operan en ambas direcciones: si existe un vuelo Quito → Latacunga, también existe Latacunga → Quito con la misma distancia en km. El grafo es ponderado de forma obligatoria, usando kilómetros reales entre ciudades de la Sierra del Ecuador.

Región geográfica: Sierra del Ecuador

Se utilizan 6 ciudades reales de la Sierra ecuatoriana con distancias reales en km:

Origen	Destino	Distancia (km)
Quito	Latacunga	89 km
Quito	Ibarra	115 km
Latacunga	Ambato	47 km
Ambato	Riobamba	51 km
Riobamba	Guaranda	65 km
Guaranda	Ambato	72 km

Requerimientos Técnicos: 10 Operaciones Implementadas

El sistema permite al usuario interactuar con el grafo a través de las siguientes 10 operaciones obligatorias, todas implementadas en la clase GrafoEcoVuelo:

#	Operación	Descripción
1	anadir_ciudad()	Inserta un nuevo aeropuerto al sistema como vértice.
2	eliminar_ciudad()	Elimina un aeropuerto y todas sus rutas conectadas (sin aristas huérfanas).
3	anadir_ruta()	Crea un vuelo directo ponderado (en km) entre dos ciudades existentes.
4	eliminar_ruta()	Cancela un vuelo directo entre dos ciudades.
5	buscar_ruta()	Verifica si existe un vuelo directo entre Ciudad A y Ciudad B.
6	buscar_ciudad()	Comprueba si un aeropuerto está registrado en el sistema.
7	destinos_directos()	Lista todas las ciudades conectadas directamente desde un origen.
8	consultar_distancia()	Devuelve el peso (km) de la ruta directa entre dos ciudades.
9	calcular_conectividad()	Calcula cuántas rutas totales se conectan a una ciudad.
10	bfs() y dfs()	BFS: recorre nivel por nivel. DFS: explora en profundidad antes de retroceder.

Módulo 1 — Clase GrafoEcoVuelo

La clase GrafoEcoVuelo implementa el grafo usando una lista de adyacencia (diccionario de diccionarios). Todas las operaciones se gestionan directamente sobre esta estructura sin librerías externas de grafos.

```
class GrafoEcoVuelo:
    def __init__(self):
        # Lista de adyacencia: { ciudad: { vecino: km } }
        self.vertices = {}

    # 1. Añadir Ciudad (Agregar Vértice)
    def anadir_ciudad(self, ciudad):
        if ciudad not in self.vertices:
            self.vertices[ciudad] = {}
            print(f"Ciudad '{ciudad}' agregada al sistema.")
        else:
            print(f"'{ciudad}' ya existe en el sistema.")

    # 2. Eliminar Ciudad (Eliminar Vértice + sus aristas)
    def eliminar_ciudad(self, ciudad):
```




```
if ciudad in self.vertices:
    del self.vertices[ciudad]
    for c in self.vertices:
        if ciudad in self.vertices[c]:
            del self.vertices[c][ciudad]
    print(f"Ciudad '{ciudad}' y sus rutas eliminadas.")
else:
    print(f"'{ciudad}' no existe en el sistema.")

# 3. Añadir Ruta (Agregar Arista Ponderada)
def anadir_ruta(self, origen, destino, km):
    if origen not in self.vertices or destino not in self.vertices:
        print("Una o ambas ciudades no existen.")
        return
    self.vertices[origen][destino] = km
    self.vertices[destino][origen] = km
    print(f"Ruta {origen} <-> {destino} ({km} km) agregada.")

# 4. Eliminar Ruta (Eliminar Arista)
def eliminar_ruta(self, origen, destino):
    if origen in self.vertices and destino in self.vertices[origen]:
        del self.vertices[origen][destino]
        del self.vertices[destino][origen]
        print(f"Ruta {origen} <-> {destino} eliminada.")
    else:
        print("La ruta no existe.")

# 5. Buscar Ruta (Verificar Arista)
def buscar_ruta(self, origen, destino):
    if origen in self.vertices and destino in self.vertices[origen]:
        print(f"Si existe ruta directa entre {origen} y {destino}.")
    else:
        print(f"No existe ruta directa entre {origen} y {destino}.")

# 6. Buscar Ciudad (Verificar Vértice)
def buscar_ciudad(self, ciudad):
    if ciudad in self.vertices:
        print(f"'{ciudad}' esta registrada en el sistema.")
    else:
        print(f"'{ciudad}' NO esta registrada en el sistema.")

# 7. Destinos Directos (Obtener Adyacentes)
def destinos_directos(self, ciudad):
    if ciudad not in self.vertices:
        print("Ciudad no existe.")
        return
    print(f"Destinos directos desde {ciudad}: {list(self.vertices[ciudad].keys())}")

# 8. Consultar Distancia (Obtener Peso)
def consultar_distancia(self, origen, destino):
    if origen in self.vertices and destino in self.vertices[origen]:
        print(f"Distancia {origen} <-> {destino}: {self.vertices[origen][destino]}
km")
    else:
        print("No existe ruta directa entre esas ciudades.")

# 9. Calcular Conectividad (Grado del Vértice)
def calcular_conectividad(self, ciudad):
    if ciudad not in self.vertices:
        print("Ciudad no existe.")
        return
    print(f"Conectividad de {ciudad}: {len(self.vertices[ciudad])} rutas directas")
```



```
# 10a. BFS - Búsqueda en Anchura
def bfs(self, inicio):
    if inicio not in self.vertices:
        print("Ciudad no existe.")
        return
    visitados = []
    cola = [inicio]
    vistos = {inicio}
    while cola:
        actual = cola.pop(0)
        visitados.append(actual)
        for vecino in self.vertices[actual]:
            if vecino not in vistos:
                vistos.add(vecino)
                cola.append(vecino)
    print(f"BFS desde {inicio}: {visitados}")

# 10b. DFS - Búsqueda en Profundidad
def dfs(self, inicio):
    if inicio not in self.vertices:
        print("Ciudad no existe.")
        return
    visitados = []
    self._dfs(inicio, set(), visitados)
    print(f"DFS desde {inicio}: {visitados}")

def _dfs(self, nodo, vistos, visitados):
    vistos.add(nodo)
    visitados.append(nodo)
    for vecino in self.vertices[nodo]:
        if vecino not in vistos:
            self._dfs(vecino, vistos, visitados)

# Mostrar red completa
def mostrar_red(self):
    print("\n--- RED AEREA ECOVUELO ---")
    for ciudad, rutas in self.vertices.items():
        print(f"    {ciudad} -> {rutas}")
    print("-" * 30)

print("Clase GrafoEcoVuelo cargada correctamente.")
```

[1] ✓ 0.0s

... Clase GrafoEcoVuelo cargada correctamente.

Módulo 2 — Datos Iniciales (Sierra del Ecuador)

Se instancia el grafo y se cargan las 6 ciudades reales con sus rutas y distancias en km:

```
grafo = GrafoEcoVuelo()

# 6 ciudades reales de la Sierra
for ciudad in ["Quito", "Latacunga", "Ambato", "Riobamba", "Guaranda", "Ibarra"]:
    grafo.anadir_ciudad(ciudad)

# Rutas con distancias reales en km
```



```
grafo.anadir_ruta("Quito", "Latacunga", 89)
grafo.anadir_ruta("Quito", "Ibarra", 115)
grafo.anadir_ruta("Latacunga", "Ambato", 47)
grafo.anadir_ruta("Ambato", "Riobamba", 51)
grafo.anadir_ruta("Riobamba", "Guaranda", 65)
grafo.anadir_ruta("Guaranda", "Ambato", 72)

grafo.mostrar_red()
```

```
Ciudad 'Quito' agregada al sistema.
Ciudad 'Latacunga' agregada al sistema.
Ciudad 'Ambato' agregada al sistema.
Ciudad 'Riobamba' agregada al sistema.
Ciudad 'Guaranda' agregada al sistema.
Ciudad 'Ibarra' agregada al sistema.
Ruta Quito <-> Latacunga (89 km) agregada.
Ruta Quito <-> Ibarra (115 km) agregada.
Ruta Latacunga <-> Ambato (47 km) agregada.
Ruta Ambato <-> Riobamba (51 km) agregada.
Ruta Riobamba <-> Guaranda (65 km) agregada.
Ruta Guaranda <-> Ambato (72 km) agregada.

--- RED AEREA ECOVUELO ---
Quito -> {'Latacunga': 89, 'Ibarra': 115}
Latacunga -> {'Quito': 89, 'Ambato': 47}
Ambato -> {'Latacunga': 47, 'Riobamba': 51, 'Guaranda': 72}
Riobamba -> {'Ambato': 51, 'Guaranda': 65}
Guaranda -> {'Riobamba': 65, 'Ambato': 72}
Ibarra -> {'Quito': 115}
```

Módulo 3 — Interfaz de Escritorio (Tkinter + Mapa en Vivo)

Descripción de la interfaz:

- Panel izquierdo: controles en tres pestañas — Ciudades, Rutas y Recorridos — más cuadro de Resultado.
- Panel derecho: mapa de la red que se dibuja y actualiza en vivo al añadir/eliminar ciudades o rutas.
- Al elegir Origen/Destino en la pestaña Rutas, ambas ciudades se resaltan y la arista entre ellas se marca si existe.
- Al ejecutar BFS/DFS, el camino recorrido se resalta sobre el mapa en tiempo real.
- Todas las operaciones usan botones, campos de texto y listas desplegables — sin input() de consola.
- Incluye Dijkstra como funcionalidad extra: si no hay vuelo directo entre dos ciudades, calcula y muestra la ruta más corta posible.

```
# — Módulo 7: Interfaz de Escritorio (Tkinter) con mapa en vivo —————
import tkinter as tk
from tkinter import ttk

import heapq
```

```
import numpy as np
from matplotlib.figure import Figure
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import matplotlib.patches as mpatches
import matplotlib.path as mpath
import matplotlib.pyplot as plt

COLOR_OK = "#1A8A60"
COLOR_ERR = "#C94A0C"
COLOR_Aviso = "#B07010"
COLOR_INFO = "#1557C0"
COLOR_ORIGEN = "#1A8A60"
COLOR_DESTINO = "#B07010"
COLOR_ARISTA = "#1557C0"
COLOR_SELEC = "#7A3FA0"

PLACEHOLDER = "- selecciona -"

# Posiciones geográficas base (Sierra Ecuador, coordenadas normalizadas 0-1)
_POS_BASE = {
    "Ibarra": (0.18, 0.88),
    "Quito": (0.42, 0.68),
    "Latacunga": (0.46, 0.46),
    "Ambato": (0.70, 0.30),
    "Guaranda": (0.22, 0.16),
    "Riobamba": (0.52, 0.06),
}

class VentanaEcoVuelo:
    def __init__(self, grafo):
        self.grafo = grafo

        # Estado de resaltado del mapa (se actualiza según la interacción del usuario)
        self._h_origen = None
        self._h_destino = None
        self._h_camino = None
        self._h_color_camino = COLOR_ARISTA

        # Posiciones ya calculadas en el mapa (se conservan entre redibujos para
        # que las ciudades no "salten" de lugar cada vez que se actualiza el mapa)
        self._pos_cache = dict(_POS_BASE)

        self.root = tk.Tk()
        self.root.title("✈ EcoVuelo - Sistema de Rutas Aéreas (Escritorio)")
        self.root.geometry("1180x720")
        self.root.minsize(980, 600)

        estilo = ttk.Style(self.root)
        try:
            estilo.theme_use("clam")
        except tk.TclError:
            pass
```



